

Diabetes Prediction Using Machine Learning Classification Approaches: A Capstone Project by Team Nabhan

A data-driven call for early detection of diabetes from the Pima Indians Women case studies



Agbaoyeadeje

Follow

20 min read · Jun 12, 2025



197



2





Three Indian women in their traditional attires (Source: Nadine Sykora on [Contiki](#))

You have probably heard about diabetes a lot of times in your lifetime. It is that ubiquitous in prevalence. According to the *International Diabetes Federation*, over 3.4 million people died as a result of diabetes in 2024. This corresponds to 9.3% of global deaths from all causes. The countries with the largest numbers of adults with diabetes aged 20–79 years in 2024 are China, India, and the USA. The *World Health Organization (WHO)* further published this grave illness as one of the leading causes of deaths among females in 2021 (8th, specifically)

Now, what exactly is diabetes? Diabetes isn't just about a condition from eating much sugar or carbs, as common myths would say.

Diabetes is a serious, chronic condition that occurs when the body cannot produce enough insulin or

cannot effectively use the insulin it does produce, leading to high levels of glucose in the bloodstream. The main types include Type 1, Type 2, and gestational diabetes. Type 1 diabetes is an autoimmune disease leading to the destruction of insulin-producing cells in the pancreas, while Type 2 is primarily associated with insulin resistance. Gestational diabetes occurs during pregnancy and typically resolves after childbirth.

In this article, we'll dive into a detailed overview of a machine learning project focused on predicting diabetes in patients. The primary objective of this work is to leverage patient health metrics to accurately predict the likelihood of diabetes, thereby contributing to early detection and improved patient outcomes.

The steps we took while undertaking this project are as follows:

1. Understanding the data
2. Gathering the Data
3. Data Cleaning and Validation
4. Exploratory Data Analysis
5. Feature Preprocessing
6. Modeling
7. Model Evaluation

8. Feature Importance

9. Test dummy data

Like neo-soul singer, Jill Scott from Philly would say,

Let's take a long walk, find a way to spark conversation, verbal elation, stimulation.

Understanding the data

I had to put on my detective glasses and dive into the healthcare world — not just as a student-professional, but also as a patient. That, to me, is what data science is really about: fully understanding the problem you're solving, both on a micro and macro level. It also requires deep empathy — keeping people at the heart of every solution you build.

Background:

A mobile health clinic wants to pre-screen patients for diabetes using basic health indicators to reduce hospital crowding and focus on at-risk individuals.

Problem Statement:

Use EDA to uncover factors that greatly influence the risk of diabetes. Develop a classification model to predict if a person has diabetes based on attributes like BMI, glucose levels, insulin levels, and age.

Gathering the data

Once the end goal was defined, the next step was to gather labeled data aligned with that objective. For this project, the dataset was sourced from Kaggle. This dataset is originally from the National Institute of Diabetes and Digestive and Kidney Diseases. In particular, all patients here are females at least 21 years old of Pima Indian heritage.

The initial step in the workflow involved importing the necessary libraries. We began with importing the required modules and libraries needed to make our analysis

```
# For numerical operations, data manipulation and array handling
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

# For importing normalization and classifier features
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import RandomizedSearchCV
from sklearn.neighbors import KNeighborsClassifier
from scipy.stats import randint, uniform
!pip install xgboost
import xgboost as xgb

# For evaluating metrics on models
from sklearn.metrics import accuracy_score, classification_report, confusion_mat

# To clear all warning dialogs
import warnings
warnings.filterwarnings('ignore')
```

Then the data was mounted on Google Drive and loaded into the notebook

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1
...	...	...	...	...	...	...	...	...	...
763	10	101	76	48	180	32.9	0.171	63	0
764	2	122	70	27	0	36.8	0.340	27	0
765	5	121	72	23	112	26.2	0.245	30	0
766	1	126	60	0	0	30.1	0.349	47	1
767	1	93	70	31	0	30.4	0.315	23	0

768 rows × 9 columns

first look of our loaded data

This lead me to thinking, what are these numerical attributes in this dataset and what do they entail?

- **Pregnancies:** Number of times pregnant
- **Glucose:** 2-hour Plasma glucose concentration in an oral glucose tolerance test (OGTT)
- **BloodPressure:** Diastolic blood pressure in(mm Hg)
- **SkinThickness:** Triceps skin fold thickness in (mm)
- **Insulin:** 2-Hour serum insulin (μ U/ml).
- **BMI:** (weight in kg/(height in m)²)
- **DiabetesPedigreeFunction:** This provides a numerical estimate of a person's genetic predisposition to diabetes based on their family history. This ranges from 0.08 to 2.42

- **Age:** Age of patient in the dataset
- **Outcome:** Class variable (0 or 1) indicating whether patient is non-diabetic (0) or diabetic (1)

```
[7] df.shape
```

```
(768, 9)
```

We had 768 observations (rows) and 9 attributes (columns) from the shape of our dataset

Data Cleaning and Validation

Before any modeling or analysis could begin, the data had to be carefully validated and cleaned — a crucial step often underestimated in data science. First, we extended the magnifying glass to check for null or missing values, duplicates or outliers

```
[6] df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 768 entries, 0 to 767  
Data columns (total 9 columns):  
#   Column                                Non-Null Count  Dtype  
---  -  
0   Pregnancies                          768 non-null    int64  
1   Glucose                              768 non-null    int64  
2   BloodPressure                        768 non-null    int64  
3   SkinThickness                       768 non-null    int64  
4   Insulin                             768 non-null    int64  
5   BMI                                  768 non-null    float64  
6   DiabetesPedigreeFunction             768 non-null    float64  
7   Age                                  768 non-null    int64  
8   Outcome                             768 non-null    int64  
dtypes: float64(2), int64(7)  
memory usage: 54.1 KB
```

Checking for null values

At first glance, all 768 rows appear non-null across every attribute. However, this doesn't necessarily mean the dataset is clean. It simply means there are no missing values represented as `NaN`. In medical datasets like this, placeholder values such as `0` can be used to represent missing data, especially in fields like glucose, insulin, or BMI, where a value of `0` is physiologically implausible.

We already observed some abnormal zeros in the dataset, and to confirm that, we did:

```
[8] df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
Pregnancies	768.0	3.845052	3.369578	0.000	1.00000	3.00000	6.00000	17.00
Glucose	768.0	120.894531	31.972618	0.000	99.00000	117.00000	140.25000	199.00
BloodPressure	768.0	69.105469	19.355807	0.000	62.00000	72.00000	80.00000	122.00
SkinThickness	768.0	20.536458	15.952218	0.000	0.00000	23.00000	32.00000	99.00
Insulin	768.0	79.799479	115.244002	0.000	0.00000	30.50000	127.25000	846.00
BMI	768.0	31.992578	7.884160	0.000	27.30000	32.00000	36.60000	67.10
DiabetesPedigreeFunction	768.0	0.471876	0.331329	0.078	0.24375	0.37250	0.62625	2.42
Age	768.0	33.240885	11.760232	21.000	24.00000	29.00000	41.00000	81.00
Outcome	768.0	0.348958	0.476951	0.000	0.00000	0.00000	1.00000	1.00

statistical analyses of numerical attributes in the dataset

We can see that even though a patient having `0` pregnancies is very much possible, it should not be the case especially in fields like glucose, insulin, or BMI, blood pressure and skin thickness.

Probing further, we made a summary table showing frequencies of these outliers

Making a DataFrame to give me more insights about my numerical values

```
summary_table = pd.DataFrame({
    'Dtype': df.dtypes,    # Dictionary is interior
    'Count': df.count(),
    'Unique': df.nunique(),
    'Null Values': df.isnull().sum(),
    'Zero Values': u
        (df[col] == 0).sum() for col in df.columns],
    'Frequent Value (and number of occurrences)': [
        f"{df[col].mode()[0]} ({df[col].value_counts().max()})" for col in df.co
    ]
})
```

summary_table

	Dtype	Count	Unique	Null Values	Zero Values	Frequent Value (and number of occurrences)
Pregnancies	int64	768	17	0	111	1 (135)
Glucose	int64	768	136	0	5	99 (17)
BloodPressure	int64	768	47	0	35	70 (57)
SkinThickness	int64	768	51	0	227	0 (227)
Insulin	int64	768	186	0	374	0 (374)
BMI	float64	768	248	0	11	32.0 (13)
DiabetesPedigreeFunction	float64	768	517	0	0	0.254 (6)
Age	int64	768	52	0	0	22 (72)
Outcome	int64	768	2	0	500	0 (500)

summary table

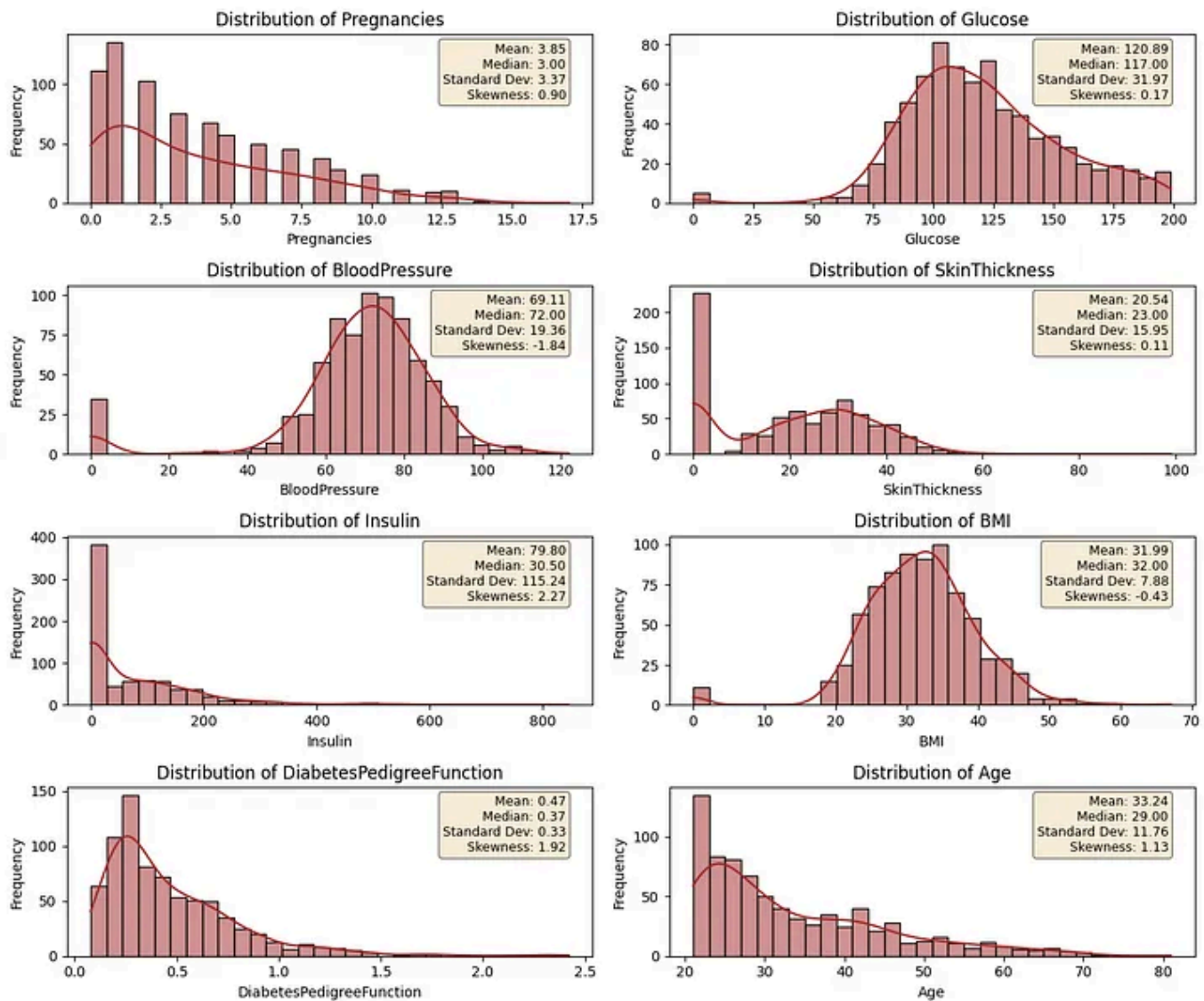
These 0 values appear a lot of times in attributes like SkinThickness and Insulin, and could greatly affect its data distribution

```
[9] df[df.duplicated()]
```

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
-------------	---------	---------------	---------------	---------	-----	--------------------------	-----	---------

checking for duplicate rows

We have no duplicate rows in our dataset



histogram distribution for attributes

The above image was to check each of the attribute distributions for their degree of skewness, so as to determine what statistical quantity to fill these 0 values with. For symmetrical distributions (e.g., SkinThickness & Glucose), We would later replace these missing values with NaN and then further fill these null values with the mean, while doing same for skewed distributions (e.g., Insulin, BloodPressure, and BMI), we used the median so as to minimize distortion.

```
columns_with_zeros = ['Glucose', 'BMI', 'Insulin', 'SkinThickness', 'BloodPressu

for column in columns_with_zeros:
    df[column] = df[column].replace(0, np.nan)

binomial_columns_with_zeros = ['Glucose', 'SkinThickness'] #Replacing the impos

for column in binomial_columns_with_zeros:
    df[column] = df[column].fillna(df[column].mean())

skewed_columns_with_zeros = ['BloodPressure', 'Insulin', 'BMI'] #Replacing the

for column in skewed_columns_with_zeros:
    df[column] = df[column].fillna(df[column].median())

df
```

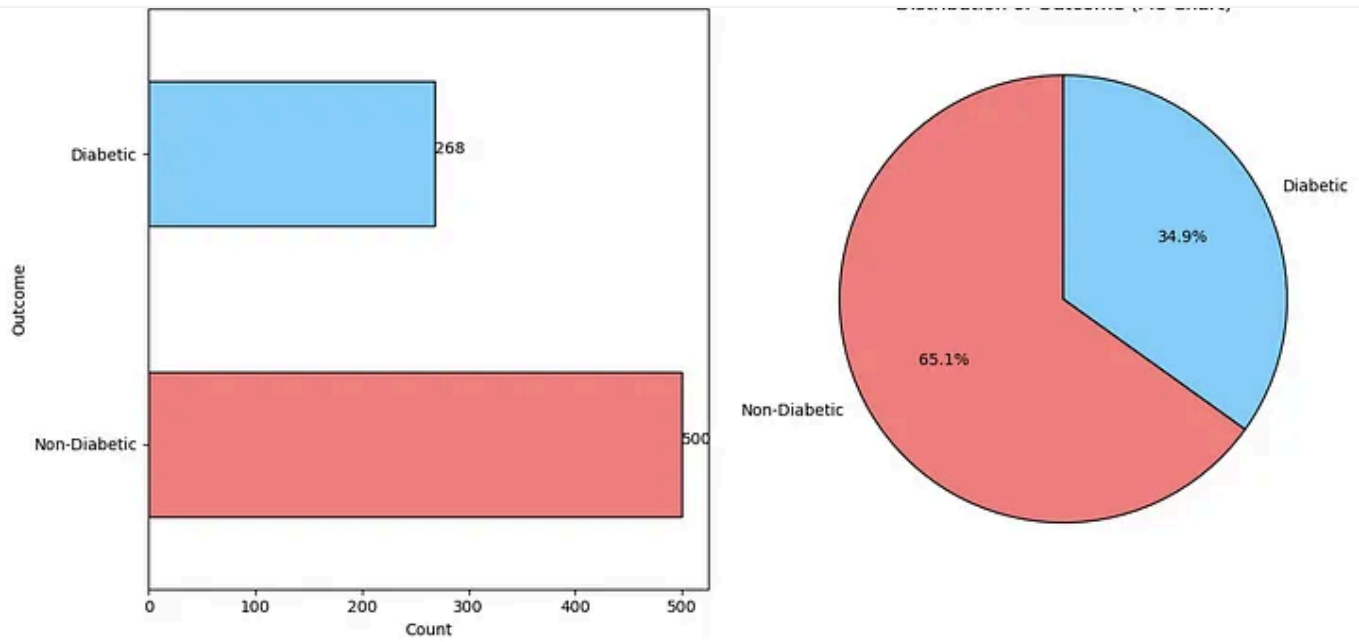
Exploratory Data Analysis

Exploratory Data Analysis (EDA) is like solving a jigsaw puzzle — it's where data scientists put their analytical instincts to the test. This is the stage where we begin to answer the *what*, *how*, *where*, and *why* behind the data using visualizations, summaries, and statistical tools. In this phase, I examined the structure, distribution, and relationships within the dataset to uncover meaningful patterns and identify potential predictors of diabetes.

As the legendary professional boxing announcer Michael Buffer would say,

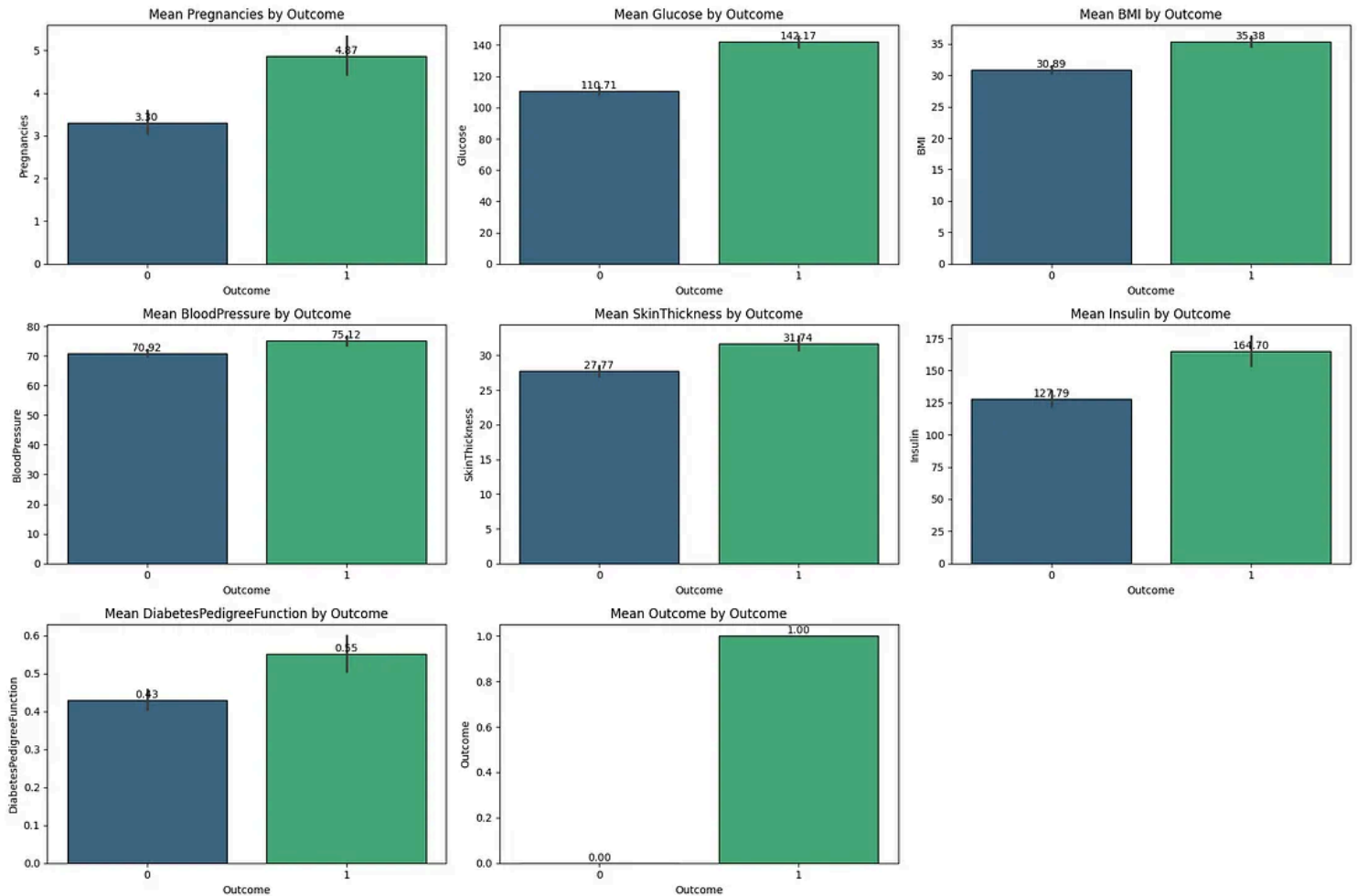
let's get ready to rumbleeee!

Open in app ↗



outcome distribution in the Pima Indians dataset

From the image above, we can see that we have 500 non-diabetic patients and 268 diabetic patients. This makes it an unbalanced distribution (this is when the target variable or class in a dataset are not represented equally) leading to potential bias or poor generalization if not handled properly



mean distribution for each attribute (coloured by outcome)

The bar charts above shows the average values of each numerical feature, separated by the 'Outcome' variable (which indicates whether a person has diabetes or not). This helped in understanding how the mean of each health metric differs between the two groups. We notice that there is clear establishment between insulin, glucose, age and BMI levels of diabetic and non-diabetic patients. This points to the aforementioned as clear indicators of diabetes in patients. The establishment in pregnancies tells us that women who have had multiple pregnancies may have a greater risk of developing gestational diabetes



correlation heatmap

The image above gives a matrix of correlations between attributes, ranging from -1 (strong negative correlation), 0 (no correlation) and +1 (strong positive correlation)

Correlations with Outcome: There's a moderately strong positive correlation between **Glucose** and **Outcome**. This is highly expected, as high blood glucose levels are a primary indicator of diabetes. This suggests that as glucose levels increase, the likelihood of a positive outcome (diabetes) also increases. A moderate positive correlation exists between **BMI** and **Outcome**. Higher BMI

is often associated with a higher risk of diabetes. A weak but positive correlation is observed between **Age** and **Outcome**. The risk of developing diabetes generally increases with age. There's a weak positive correlation between **Pregnancies** and **Outcome**. This might relate to gestational diabetes or the fact that multiple pregnancies could be associated with a higher risk of developing type 2 diabetes later.

```
# Making categorical attributes for Age, BMI, Blood Pressure - based on reliable

df['AgeGroup'] = pd.cut(df['Age'], bins=[20, 30, 40, 50, 60, 100], labels= ['You
df['BMI_Group'] = pd.cut(df['BMI'], bins=[0, 18.5, 25, 30, 100], labels=['Underw
df['BloodPressure_Group'] = pd.cut(df['BloodPressure'], bins=[0, 60, 80, 90, 120
df['Outcome_Group'] = df['Outcome'].map({0: 'No Diabetes', 1: 'Diabetes'})

df['Glucose_Category'] = pd.cut(df['Glucose'], bins=[0, 140, 200], labels=['Norm

df[['Glucose', 'Glucose_Category']]

# Define Insulin categories based on the provided ranges
df['Insulin_Category'] = pd.cut(df['Insulin'],
                                bins=[0, 30, 100, 150, 1000], # Increased upper
                                labels=['< 30 (Possible Deficiency)', '30-100 (N
                                right=False) # Use right=False to make the bins e

# Define bins for DiabetesPedigreeFunction
bins = [0, 0.5, 1.0, 1.5, df['DiabetesPedigreeFunction'].max()]
labels = ['0-0.5', '0.5-1.0', '1.0-1.5', '>1.5']

# Create a new column for the DPF bins
df['Pedigree_Bin'] = pd.cut(df['DiabetesPedigreeFunction'], bins=bins, labels=la
```

To enhance interpretability and uncover deeper patterns in the data, I transformed several continuous health metrics into clinically meaningful categories based on established healthcare stratifications. This step allows

for more intuitive analysis and visualization, especially when comparing diabetes outcomes across population segments.

- **Age** was binned into life-stage groups: *Young Adult (21–30)*, *Early Middle Age (31–40)*, *Late Middle Age (41–50)*, *Early Senior (51–60)*, and *Senior (61+)*.
- **BMI** values were binned using standard WHO classifications: *Underweight (<18.5)*, *Normal (18.5–24.9)*, *Overweight (25–29.9)*, and *Obese (30+)*.
- **Blood Pressure** was binned into clinical ranges: *Low*, *Normal*, *Pre-hypertension*, and *Stage 1–2 Hypertension* based on diastolic blood pressure cutoffs.
- The binary **Outcome** variable was mapped to more readable labels — *Diabetes* and *No Diabetes*, for better clarity in charts and tables.
- **Glucose** was binned into two primary diagnostic ranges:
 1. Normal (<140 mg/dL)
 2. Impaired Glucose Tolerance (140–199 mg/dL).

These thresholds mirror standard guidelines used in 2-hour oral glucose tolerance tests (OGTT), helping identify individuals who may be prediabetic.

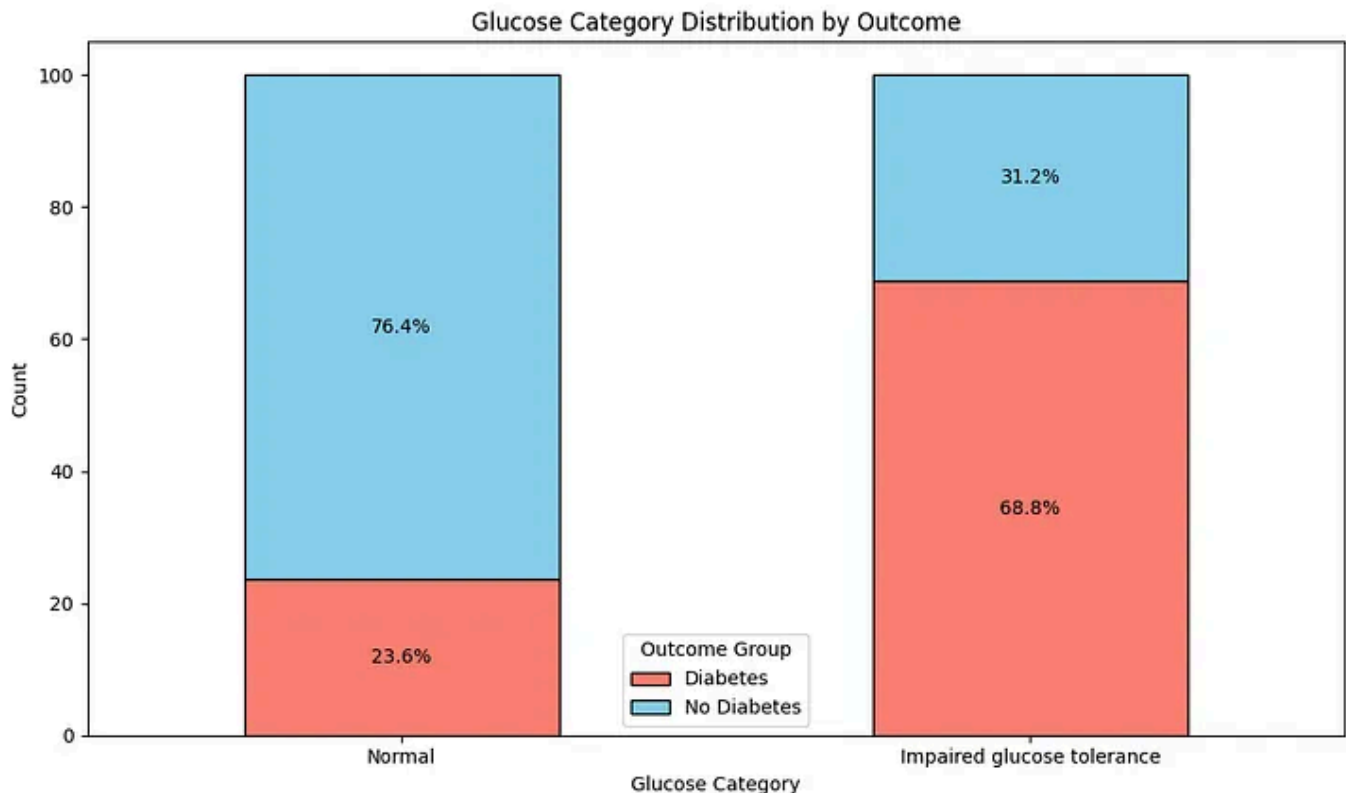
- **Insulin** levels were binned into four categories based on clinical interpretation of insulin sensitivity and resistance:
 1. < 30 $\mu\text{U/mL}$ — Possible insulin deficiency,
 2. 30–100 $\mu\text{U/mL}$ — Normal 2-hour serum insulin response
 3. 100–150 $\mu\text{U/mL}$ — Early signs of insulin resistance

4. 150 μ U/mL — Significant insulin resistance.

The bins were defined to reflect both diagnostic ranges and observed distributions in the dataset. Using `right=False` ensured that upper bounds of each category did not overlap

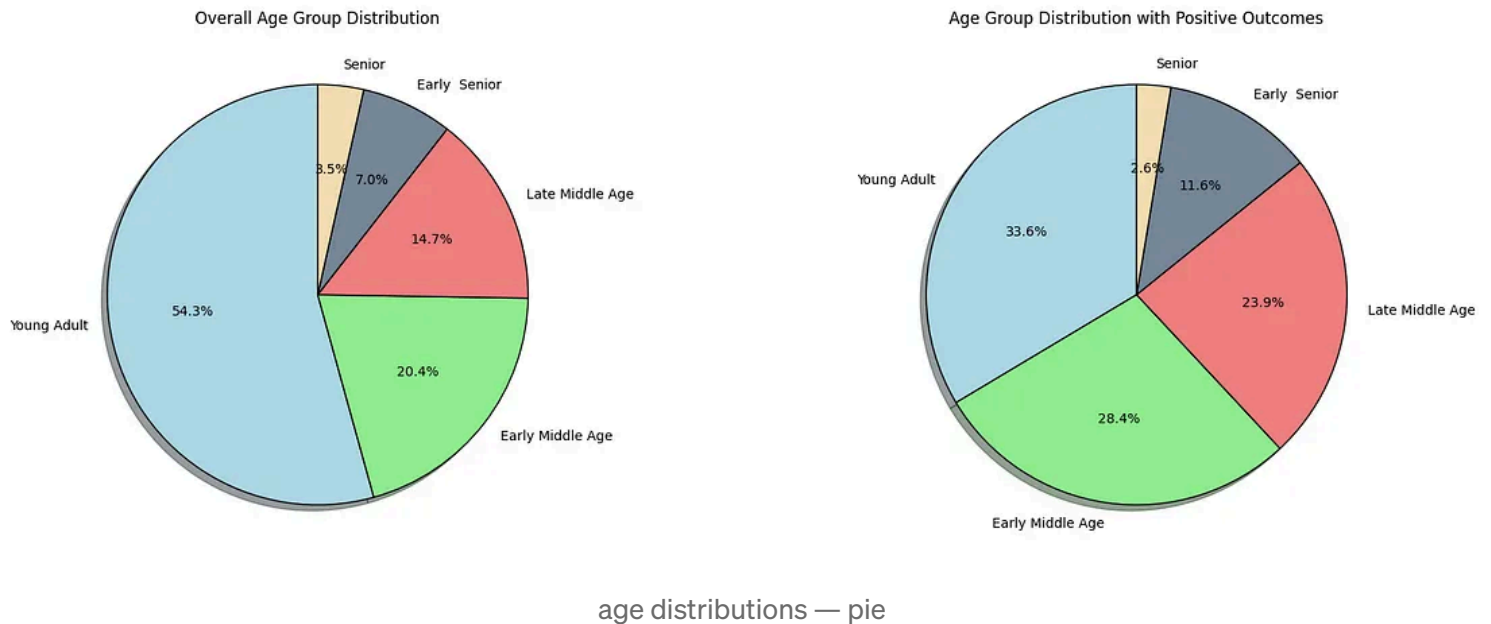
- While the **DiabetesPedigreeFunction** isn't a clinical measure in the traditional sense, it helped give some insights into a patient's genetic predisposition to diabetes based on family history. To analyze this feature more intuitively, I segmented the continuous DPF values into four categorical bins. These bins help highlight how increasing hereditary risk correlates with diabetes prevalence in the dataset.

Now, getting proper into category charting

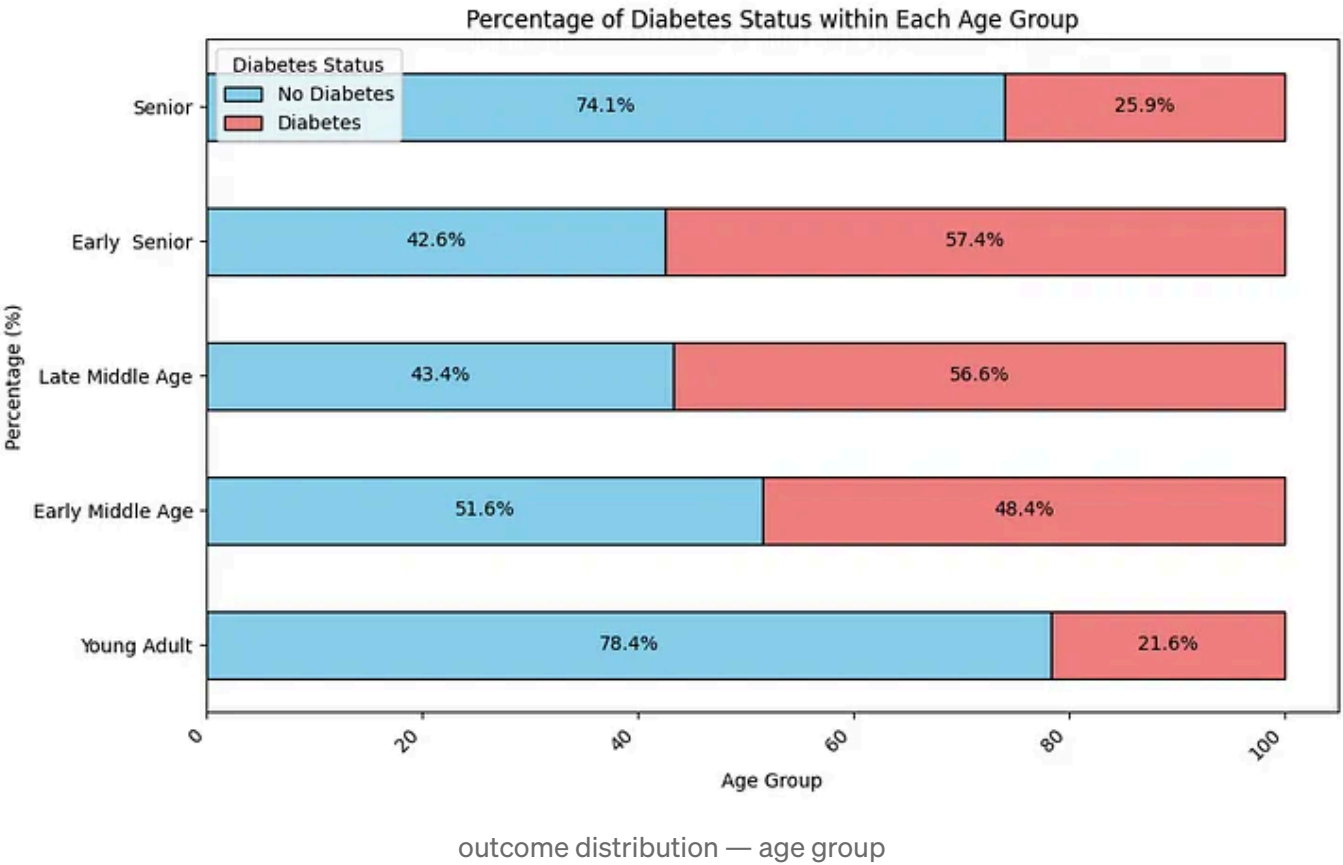


glucose category by outcome

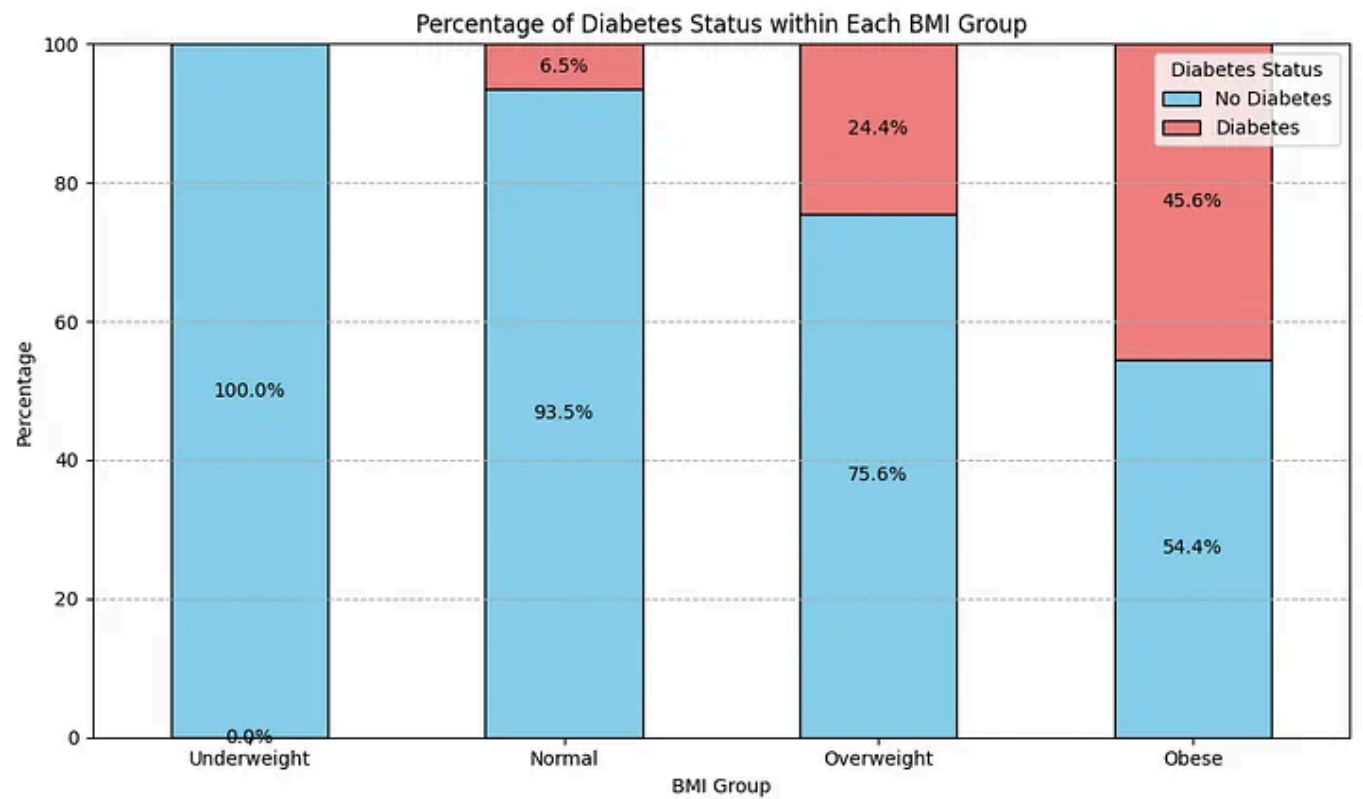
This stacked bar chart gives percentages of positive and negative outcomes of diabetes among both grouped blood glucose categories. It tells us there is a far greater percentage of diabetic patients with impaired glucose levels. Hence, this reinstates our hypothesis that glucose is a key determinant in eventual diabetes outcomes. Regular glucose testing should be taken at all age levels, even if the patient is not diabetic yet



From both pie charts, seniors are not adequately represented in our dataset, so we can take analysis from this age group with a pinch of salt

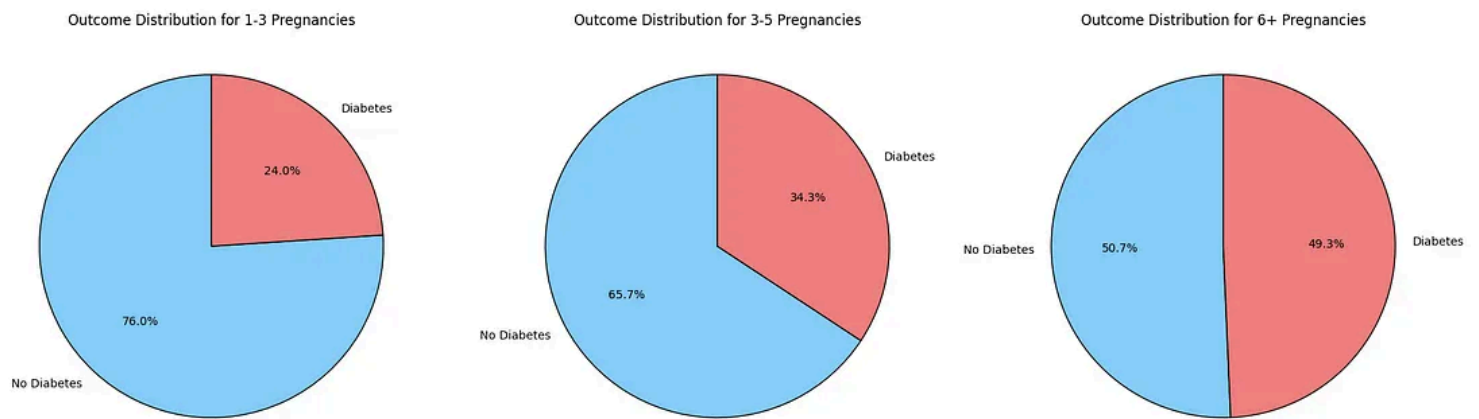


This horizontal bar chart gives insights of diabetes status distribution among each age group. From this, we can see that those with more risk of having diabetes are between Early Middle Age (31–40) and Early Seniors (51–60). Hence, early testing and monitoring at a threshold of 40 years is heavily recommended



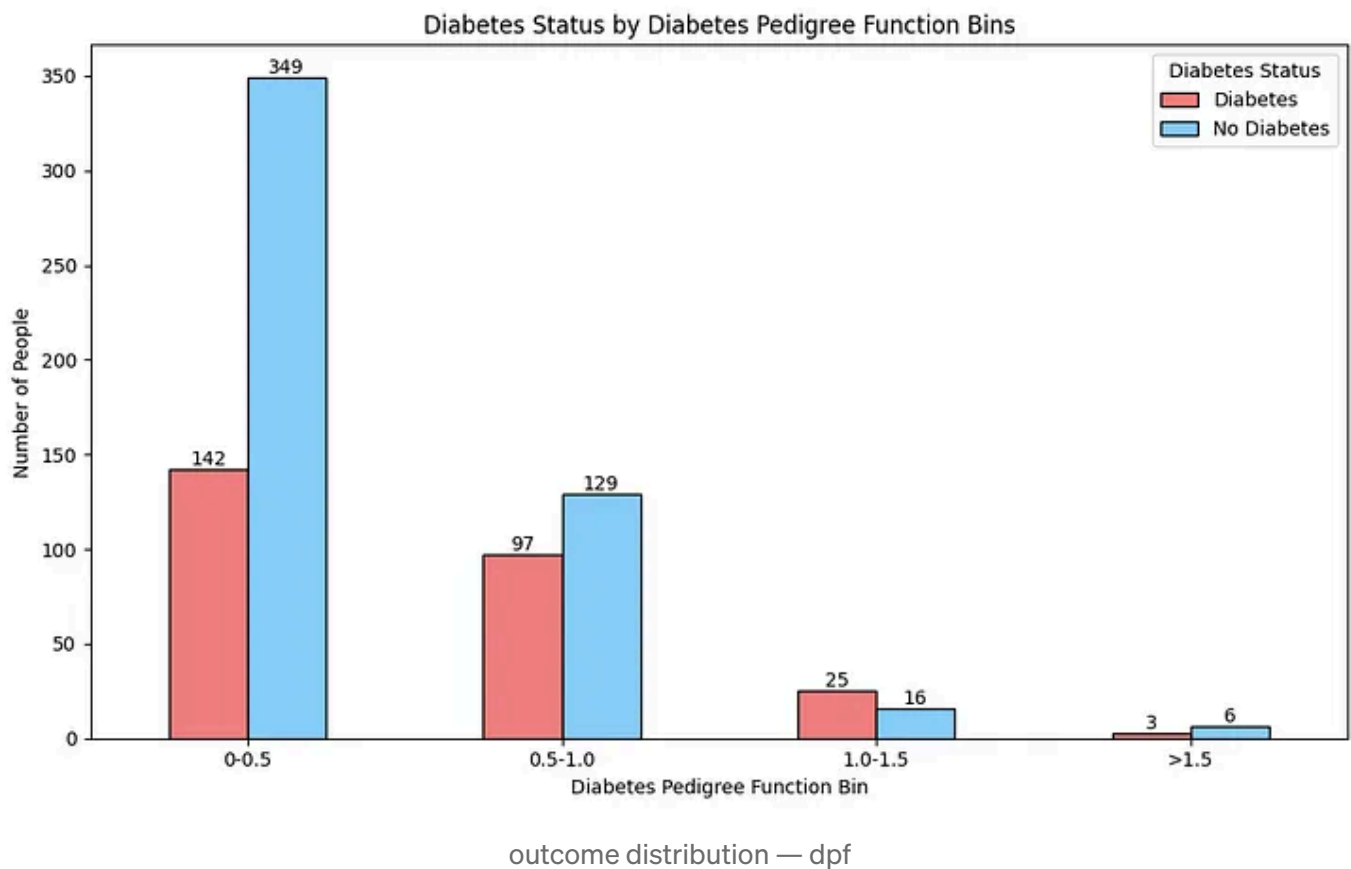
outcome distribution — bmi group

Here, we can see that those in the obese BMI class are the most liable to diabetes. Hence, aggressive weight management strategies for patients is the most viable solution here

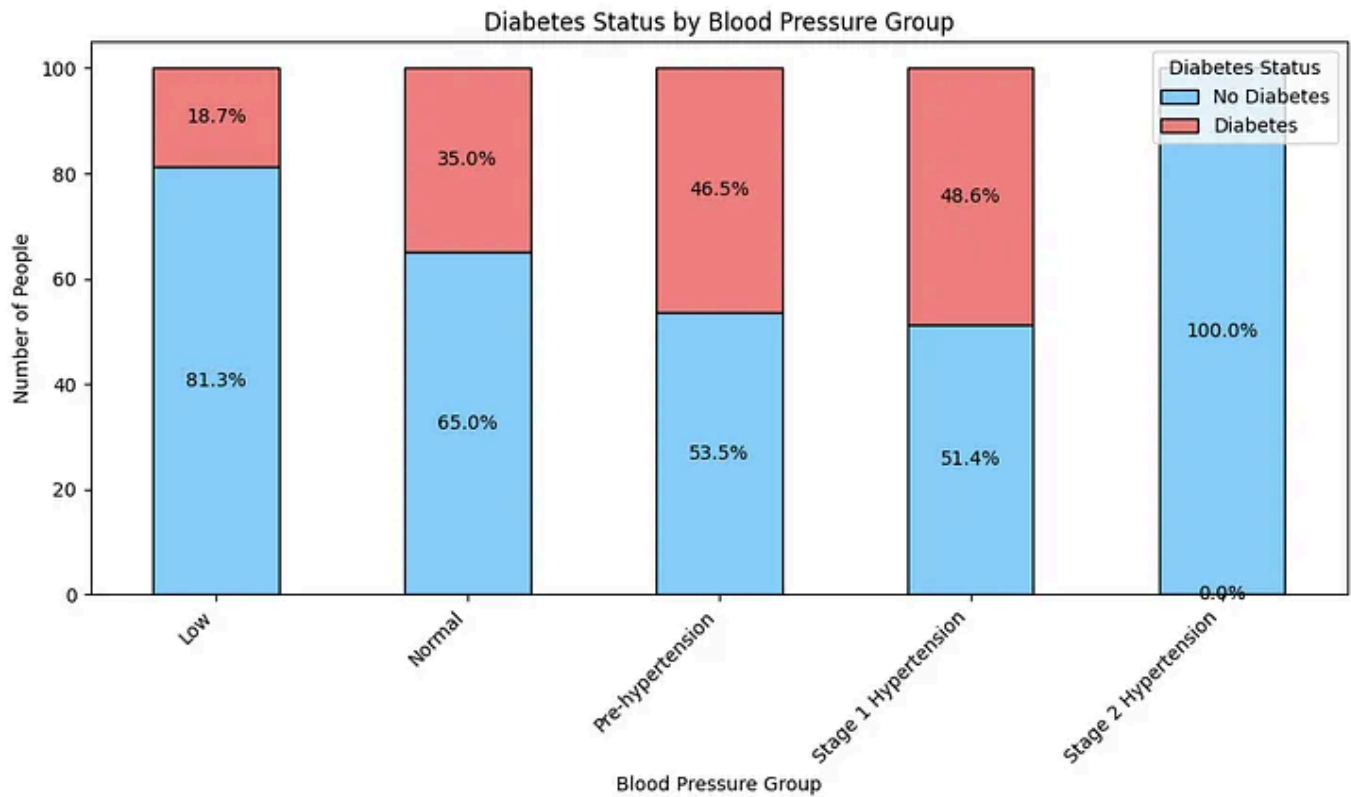


outcome distribution — binned pregnancies

There appears to be some correlation between the number of pregnancies and the prevalence of diabetes. As the number of pregnancies increases, the proportion of women diagnosed with diabetes also increases significantly. The further tells us that women have had multiple pregnancies may have a greater risk of developing gestational diabetes

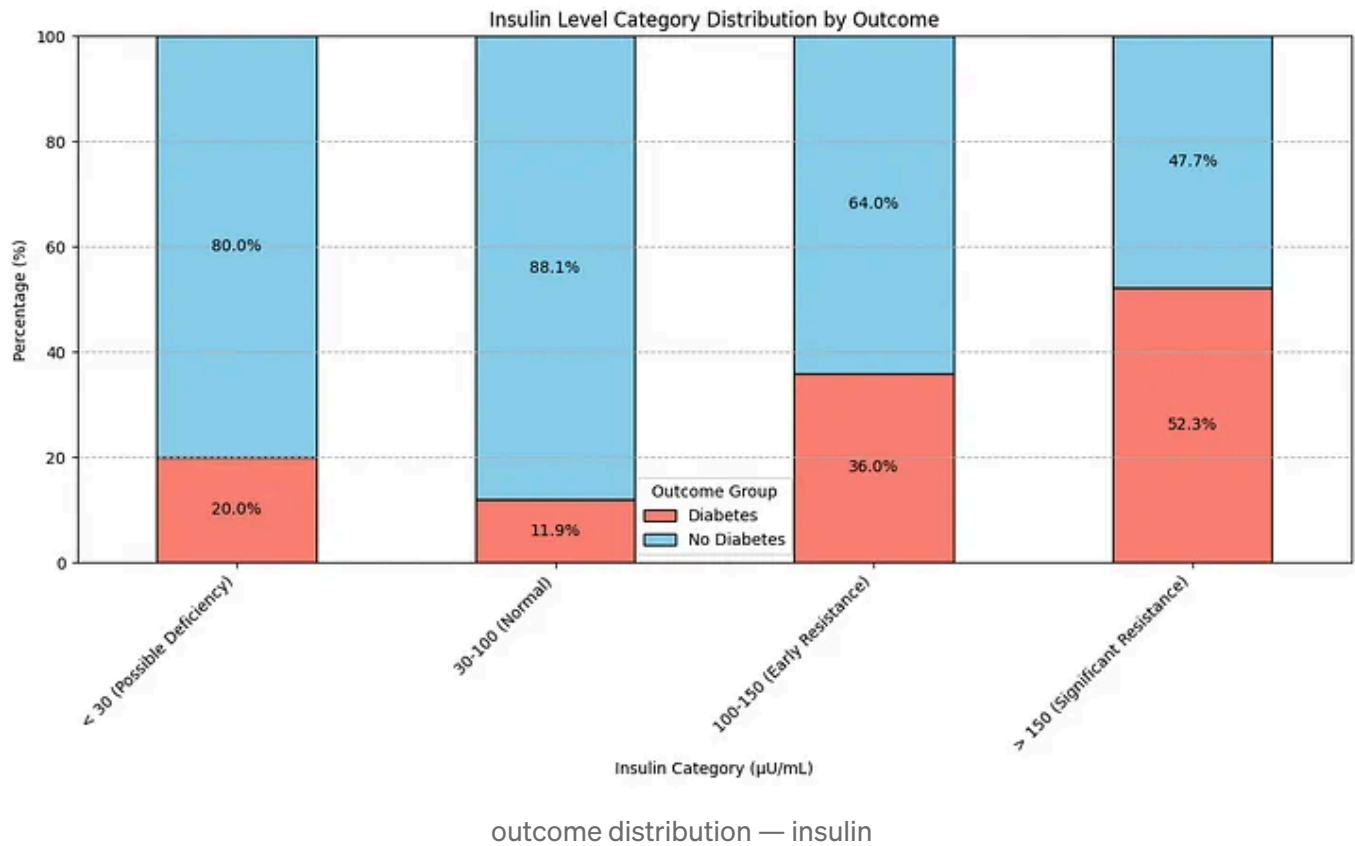


As we can see from the multiple bar chart above, we tend to have greater relative proportions of diabetic patients among higher pedigree functions. This also calls to fact that genetic predisposition might be an influence

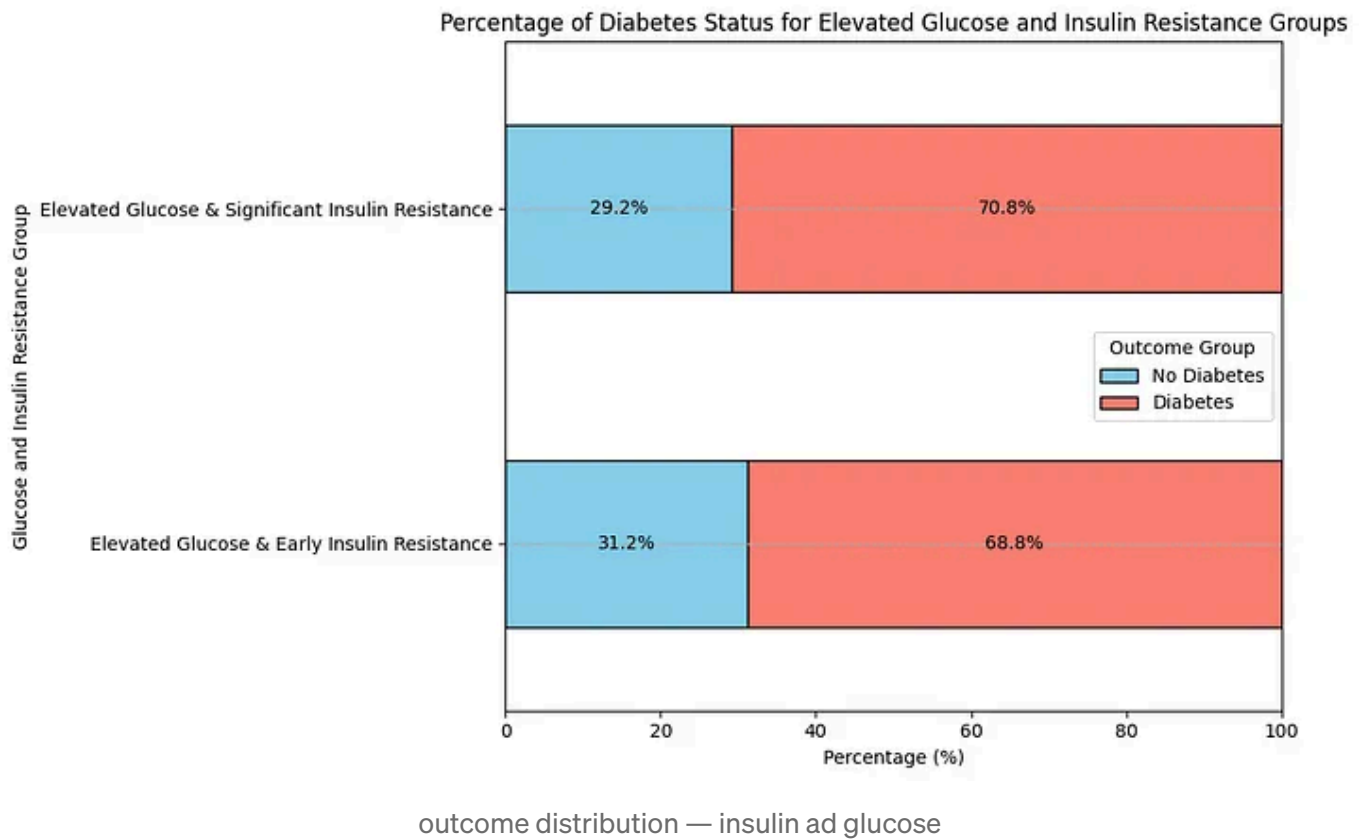


Just one observation got Stage 2 hypertension (not enough data)

From my background of understanding the data, I learnt that if you have diabetes, you are twice as likely to have high blood pressure. This looks more like an after-effect of diabetes than a cause. Diabetes causes damage by scarring the kidneys, which in turn leads to salt and water retention, which in turn raises blood pressure. Hence, we can see more pronounced effects of diabetes in patients with pre-hypertension and Stage 1 hypertension



Insulin resistance is often a precursor to impaired levels of glucose in the bloodstream. This means that individuals with significant resistance are the most prone to diabetes, where more than half of women with severe resistance are diabetic. This can be solved by regular testing of patients' insulin levels. 100mU/mL should be the flag measure for further monitoring and processes taken to ensure early prevention



Because Glucose and Insulin are hand-shook attributes, we used masking functions, operators and loc addressing to combine information from the previously created **Glucose_Category** and **Insulin_Category** columns. This identifies specific subgroups of patients based on their glucose and insulin levels and focuses only on patients who fall into these defined elevated glucose and insulin resistance groups.

Here, the figures are staggeringly high, there's a very strong chance a patient will be diabetic if they fall between these elevated ranges

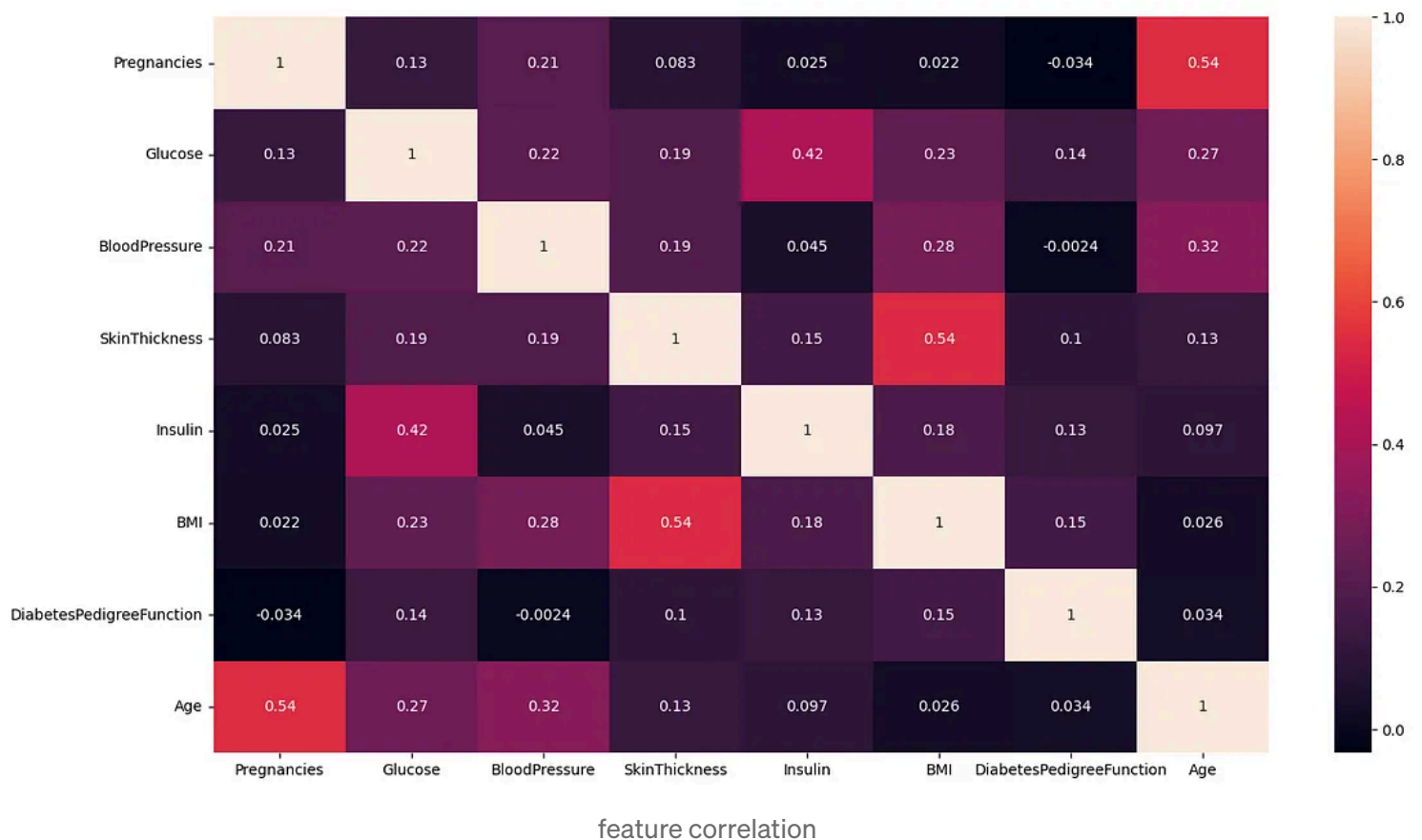
Feature pre-processing

After the whole EDA, the data needs to be prepared for modeling. First, we prepare the features (independent variables) for training a machine learning model. In supervised learning, we separate the dataset into features (the

data used to make predictions) and the target variable (what we want to predict).

```
X = df.drop(columns=df.select_dtypes(exclude=np.number))
X.drop(columns=['Outcome'], inplace=True)
X
```

```
y = df['Outcome']
y
```



Correlations Among Independent Features: A relatively strong positive correlation is present between **Glucose** and **Insulin**. This is biologically

sound, as insulin is released in response to glucose levels to regulate them.

SkinThickness and **BMI** is among one of the strongest correlations among the independent features. This makes sense as skin fold thickness measurements are often used in calculating or are related to **BMI**.

Pregnancies and **Age** also connote a strong positive correlation. This is logical, as women generally have more pregnancies as they get older.

Weak/Negative Correlations: Notice some very weak or near-zero correlations, such as **BloodPressure** and **Insulin** (0.045) or **DiabetesPedigreeFunction** and **BloodPressure** (-0.0024). These low values suggest that these features are largely independent of each other in a linear fashion {e.g. **Pregnancies** and **DiabetesPedigreeFunction** (0.034)}

Feature Scaling

```
scaler = StandardScaler()

scaledX = scaler.fit_transform(X)
```

Standardization is a common preprocessing step in machine learning to ensure that features with larger values do not disproportionately influence the model. This is possible by transforming the data such that its mean is 0 and its standard deviation is 1.

Feature Metrics

		PREDICTED	
		Positive	Negative
ACTUAL	Positive	True Positive (TP)	False Negative (FN)
	Negative	False Positive (FP)	True Negative (TN)

understanding a confusion matrix

Again, you remember when we said this dataset is unbalanced? Well, your model might tend to favor the majority class, and even if you get high accuracy, it might yield poor performance where it matters. A 95% accuracy may hide the fact that the model **never** correctly predicts the minority class (the important one) — and this could prove catastrophic when it actually matters. So instead of focusing on accuracy, we would evaluate our metrics on these models namely: **precision, accuracy, F-1 score and AUC (Area Under the Curve)**

Precision is: “Of all the times the model said ‘Diabetic’, how many were actually Diabetic?” — if your model said 10 people are diabetic and only 6 people really do, your precision is 0.6 (this will also exist for the non-diabetic class)

Precision = $(\text{True Positives}) / (\text{False Positives} + \text{True Positives})$

Recall is: “Of all the actual diabetic cases, how many did the model catch?” — this will also exist for the non-diabetic class)

Recall = $(\text{True Positives}) / (\text{False Negatives} + \text{True Positives})$

F1 Score is: “What’s the balance between precision and recall?” — this is usually the best metric for unbalanced datasets. It is the harmonic mean of both precision and recall. For your F1 score to be high, your recall and precision must be high

AUC is: “How well can the model separate the classes (e.g. Yes vs No)?” — this value is between 0 and 1. This tends to rank the overall performance of the model

Train-test split validation

To evaluate the performance of a machine learning model, it’s essential to split the dataset into two parts: the training set that is used to train the model, and the test set that is used to evaluate how well the model performs on unseen data. We allocate 20% of the dataset to the test set and 80% to the training set. We use Scikit-learn’s train-test split to perform this.

```
X_train, X_test, y_train, y_test = train_test_split(scaledX, y, test_size=0.2, r
```

Modelling

Because this is a classification model, we will use classification models such as Decision Tree, Random Forest, etc.

We started with Logistic Regression:

```
# Training the model to allow it make predictions based on the input
log = LogisticRegression()
log.fit(X_train, y_train)
```

```
# Use trained log model to make predictions on the test dataset
log_preds = log.predict(X_test)
```

➡ Accuracy Score for Logistic Regression: 0.7532467532467533

	precision	recall	f1-score	support
0	0.80	0.83	0.81	99
1	0.67	0.62	0.64	55
accuracy			0.75	154
macro avg	0.73	0.72	0.73	154
weighted avg	0.75	0.75	0.75	154

Evaluation: Accuracy Score and Classification Report for Logistic Regression

Random Forest Classification

Now, we try Random Forest

```
forest = RandomForestClassifier(random_state=56) #For easy reproducibility
forest.fit(X_train, y_train) # Train model

forest_preds = forest.predict(X_test) #Test model on unseen data
```

➡ Accuracy Score for Random Forest Classifier: 0.7662337662337663

	precision	recall	f1-score	support
0	0.82	0.82	0.82	99
1	0.67	0.67	0.67	55
accuracy			0.77	154
macro avg	0.75	0.75	0.75	154
weighted avg	0.77	0.77	0.77	154

Evaluation: Accuracy Score and Classification Report for Random Forest

Decision Tree Classifier

Decision tree next:

```
#Initialize the decison tree classifier
tree = DecisionTreeClassifier(random_state=56)

# Fit the model to the training data
tree.fit(X_train, y_train)

# Make predictions on the test data
tree_preds = tree.predict(X_test)
```

➡ Accuracy Score for Decision Tree Classifier: 0.6948051948051948

	precision	recall	f1-score	support
0	0.77	0.75	0.76	99
1	0.57	0.60	0.58	55
accuracy			0.69	154
macro avg	0.67	0.67	0.67	154
weighted avg	0.70	0.69	0.70	154

Evaluation: Accuracy Score and Classification Report for Decision Tree

KNN Classifier

K-Nearest Neighbors classifier next:

```
# Initialize the KNN classifier
knn = KNeighborsClassifier(n_neighbors=25) # You can choose the number of neighbors

# Fit the model to the training data
knn.fit(X_train, y_train)

# Make predictions on the test data
knn_preds = knn.predict(X_test)
```

➡ Accuracy Score for K-Nearest Neighbors Classifier: 0.7597402597402597

	precision	recall	f1-score	support
0	0.80	0.84	0.82	99
1	0.68	0.62	0.65	55
accuracy			0.76	154
macro avg	0.74	0.73	0.73	154
weighted avg	0.76	0.76	0.76	154

Evaluation: Accuracy Score and Classification Report for K-Nearest Neighbors Classifier

XGBoost Classifier

```
# Create an XGBoost classifier
xgb_model = xgb.XGBClassifier(random_state=42)

# Train the model
xgb_model.fit(X_train, y_train)

# Make predictions
xgb_preds = xgb_model.predict(X_test)
```

```
[116] # Evaluate the XGBoost model
print(f"Accuracy Score for XGBoost Classifier: {accuracy_score(y_test, xgb_preds)}\n")
print("Classification Report for XGBoost Classifier:")
print(classification_report(y_test, xgb_preds))
```

➞ Accuracy Score for XGBoost Classifier: 0.7402597402597403

Classification Report for XGBoost Classifier:

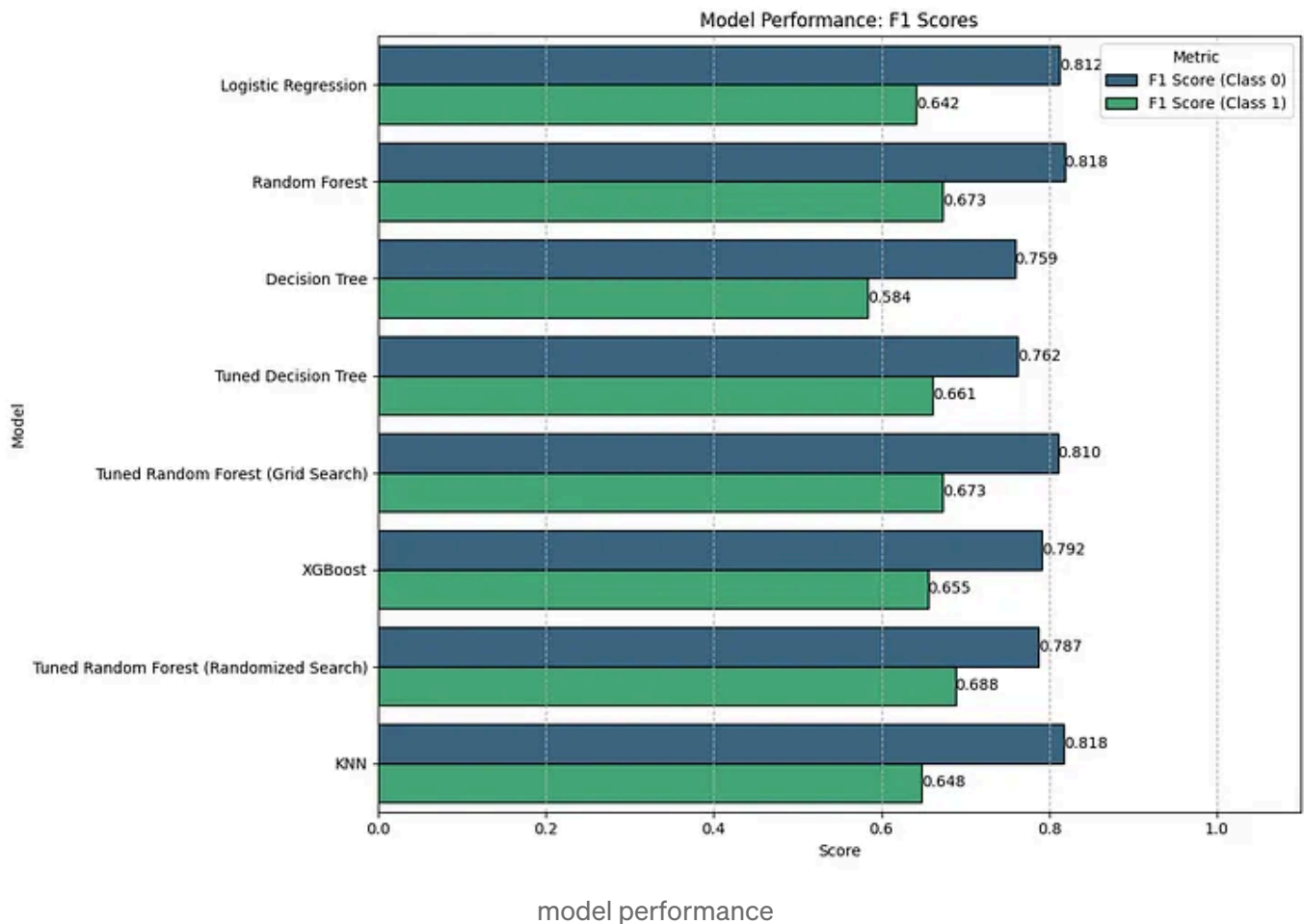
	precision	recall	f1-score	support
0	0.82	0.77	0.79	99
1	0.62	0.69	0.66	55
accuracy			0.74	154
macro avg	0.72	0.73	0.72	154
weighted avg	0.75	0.74	0.74	154

Evaluation: Accuracy Score and Classification Report for xgbclassifier

Hyperparameter Tuning

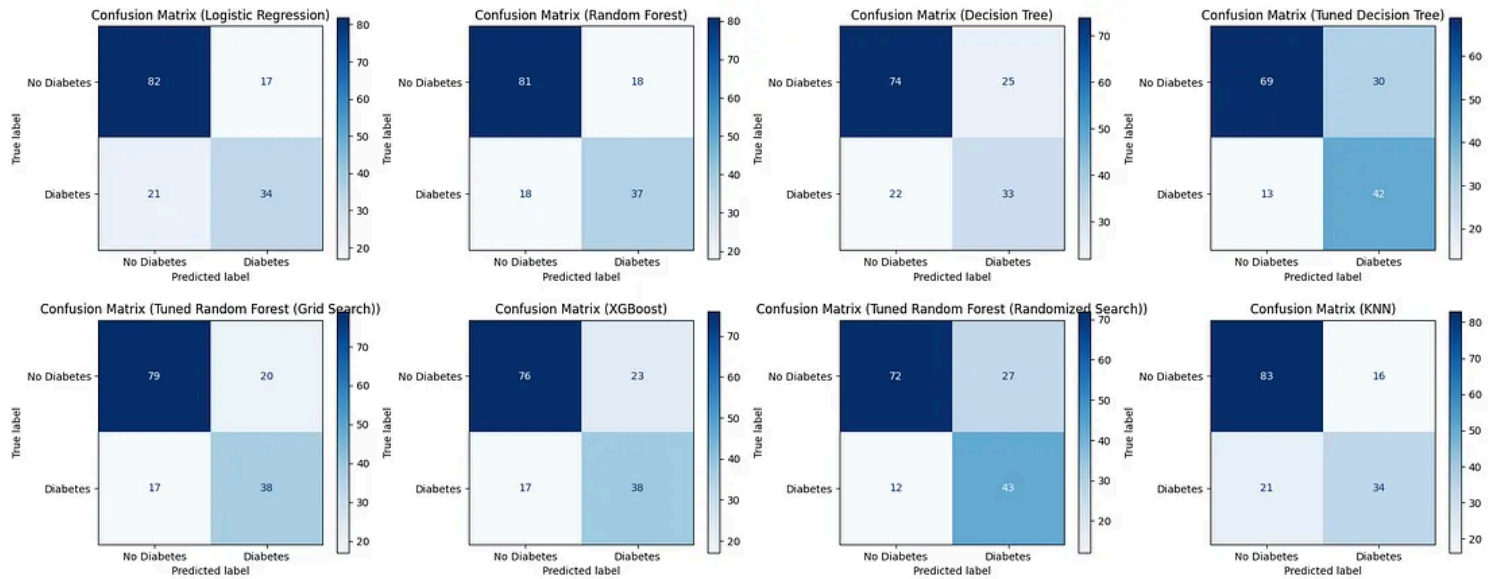
We performed GridSearch and RandomizedSearchCV for Random Forest and Decision Tree models to hopefully obtain better F1 scores during evaluation.

Here's how all the models faired eventually:



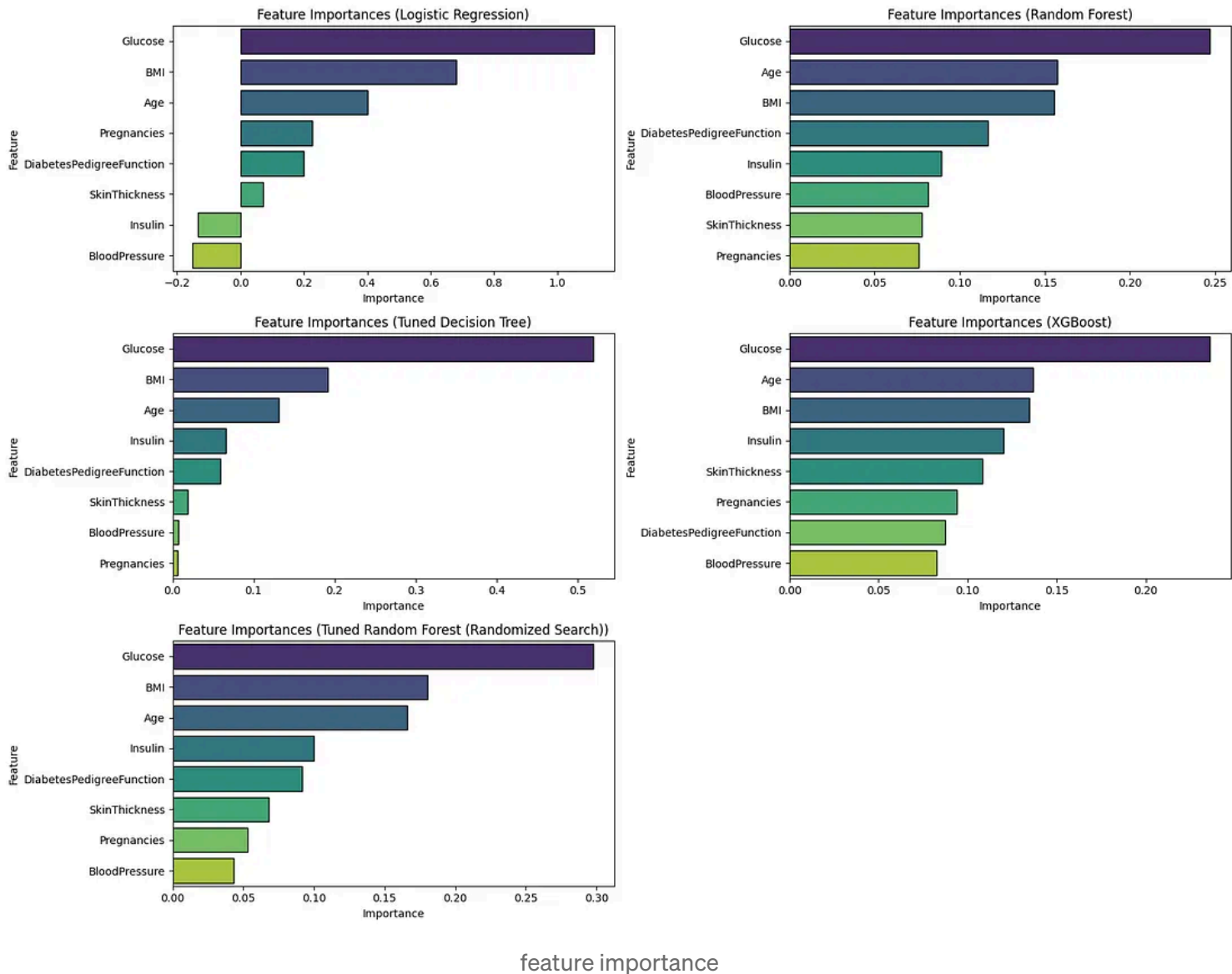
The Tuned Random Forest (Randomized Search) performed best for the minority outcome, and KNN performed best for the majority outcome. We chose to deploy the Tuned Random Forest to test our dummy data anyway, but that's in a bit.

Here is the 2*4 grid of confusion matrices for all models used



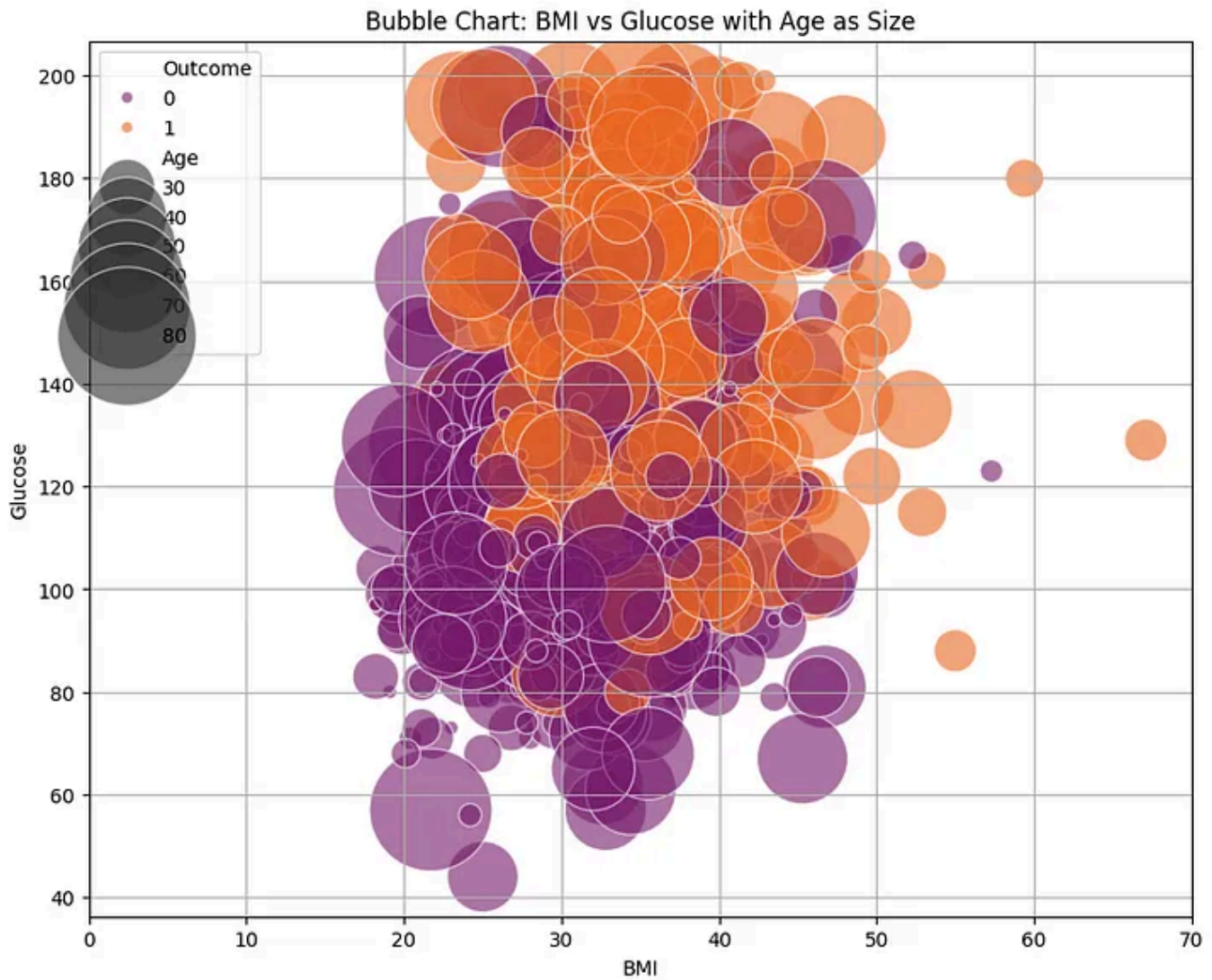
Feature Importance

This helps give us a ranking of attributes used by each model in training and testing to determine or select between classes



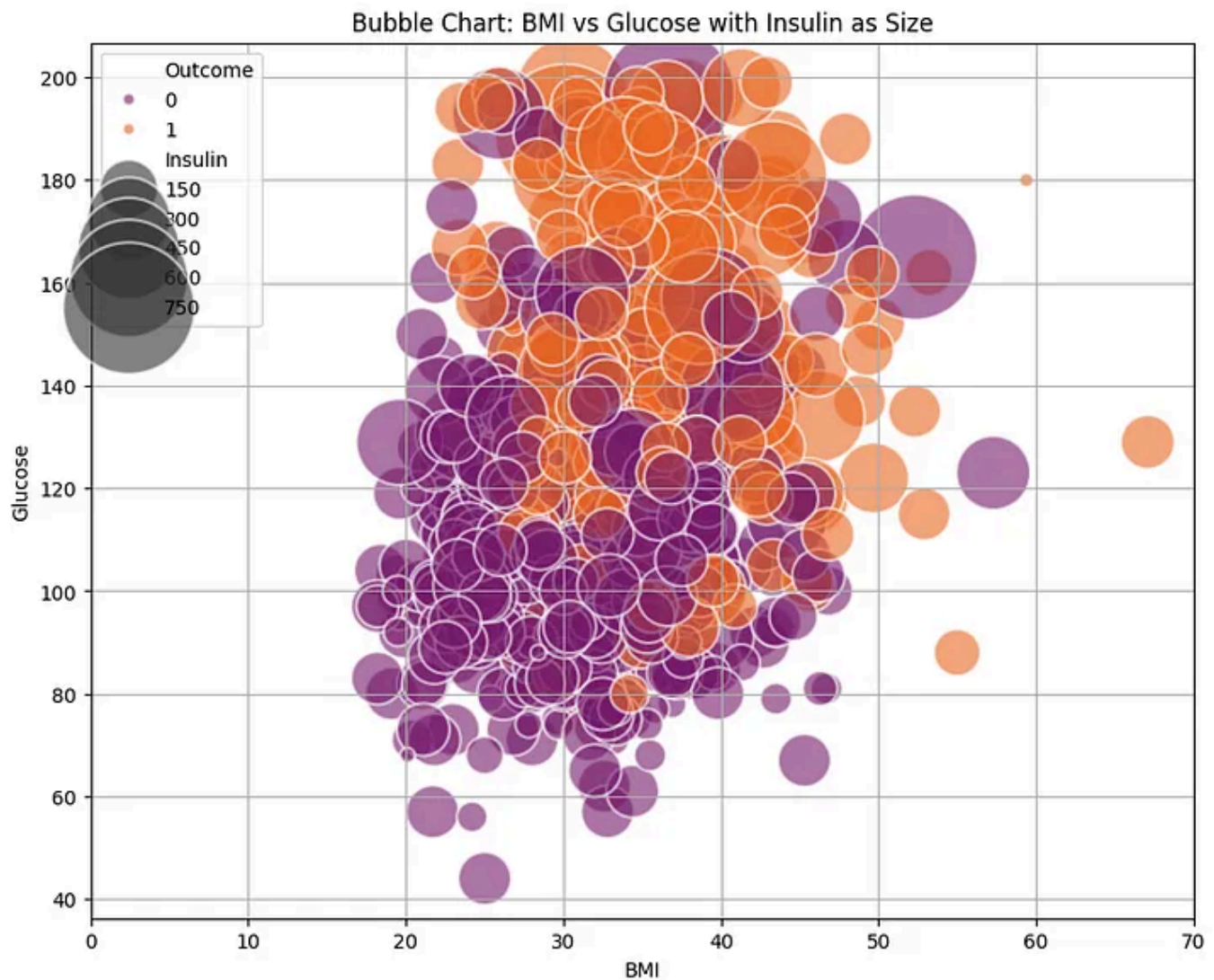
One thing is certain from this: Glucose is the number one determinant for diabetes prediction in patients. BMI, Age, and Insulin are also key attributes that contribute to outcome prediction.

Let’s test that out a bit:



Do you see this bubble chart? This tells us that there is a cluster of large orange (Outcome = 1) bubbles at higher glucose and BMI levels, suggesting that older individuals with higher BMI and glucose values are more likely to have a positive outcome. You'll notice that many large purple circles are lower on the glucose axis. This means they might be older and heavier — but their glucose levels are not high, so they're not diabetic.

- *Big orange (above)* = high BMI + high glucose + older = higher risk
- *Big purple (under)* = may be old or have high BMI, but have normal glucose or other protective factors



insulin resistance bubble chart

The circles on the topmost quadrant are mostly huge and orange. This tells us that insulin resistance and elevated glucose (see: bar-chart above) is not a good look. It almost certainly yields diabetic, or a ticking time-bomb at least. Majority of the negative outcomes here have normal glucose and insulin levels

Now here's the jaw-dropper! Because I like risk analysis, we like measuring chances in terms of odds. we built a simple risk scoring system that assigns +1 if BMI > 30, +1 if Glucose > 140, +1 if Age > 40, +1 if Insulin > 150, and +1 if number of pregnancies > 3. We calculated the diabetes rate for each score

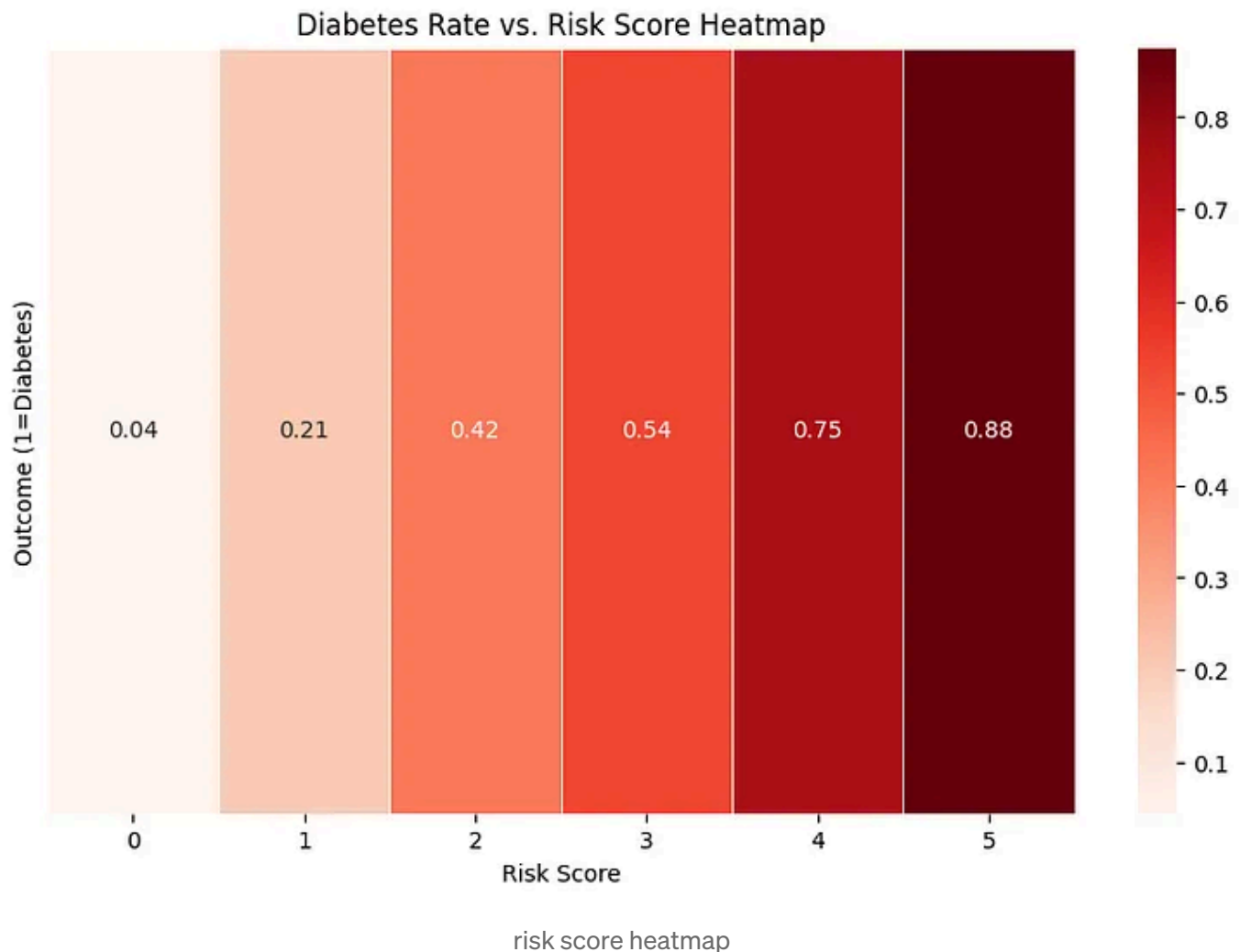
and tabled them on a heatmap that ranges from 0–1 (the closer to 1 the greater a patient's risk of having diabetes is — based on attributes fulfilled)

```
# Apply the risk scoring logic based on most important features to the DataFrame
def calculate_risk_score(row):
    score = 0 # Initialize 0 and increment by condition
    if row['BMI'] > 30:
        score += 1
    if row['Glucose'] > 140:
        score += 1
    if row['Age'] > 40:
        score += 1
    if row['Insulin'] > 150:
        score += 1
    if row['Pregnancies'] > 3:
        score += 1
    return score

df['RiskScore'] = df.apply(calculate_risk_score, axis=1)

# Calculate the diabetes rate for each risk score
risk_diabetes_rate = df.groupby('RiskScore')['Outcome'].mean().reset_index()

# Create the heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(risk_diabetes_rate.set_index('RiskScore').T, annot=True, cmap='Reds')
plt.title('Diabetes Rate vs. Risk Score Heatmap')
plt.xlabel('Risk Score')
plt.ylabel('Outcome (1=Diabetes)')
plt.yticks([]) # Hide the y-axis label as it's just 'Outcome (1=Diabetes)'
plt.show()
```



Even just as little as 3 thresholds exceeded, a patient is more or less likely to be diabetic. 4 thresholds exceeded is a huge calamitous red flag we want to quickly find an intervention to, and 5 is almost definitely diabetic

Testing Dummy Data

We passed a dummy dataset to a model to test if this patient is positive or negative. We pass the data as an array in the same order as it is in the data dictionary, scale the data, perform the prediction using the model deployed, and assess probabilities of certainty for both outcomes. Look at the code below:

```
# Example patient data as a NumPy array
# The order of features were ensured to match the training data
# The features in the training data (X) were:
# ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI',

# Example patient data (replace with actual patient data)
# This is just a placeholder array with 8 values.
# You need to provide the actual patient data here.
patient_data_array = np.array([[2, 130, 70, 30, 128, 35.0, 0.82, 43]])

# Scale the patient data using the same scaler fitted on the training data
scaled_patient_data = scaler.transform(patient_data_array)

# Perform the prediction using the Randomized Search model
prediction = best_tree_model.predict(scaled_patient_data)

# The prediction result is a numpy array, access the first element
prediction_result = prediction[0]

# Interpret the prediction
if prediction_result == 1:
    print("Prediction: The patient is likely to have diabetes.")
else:
    print("Prediction: The patient is likely not to have diabetes.")

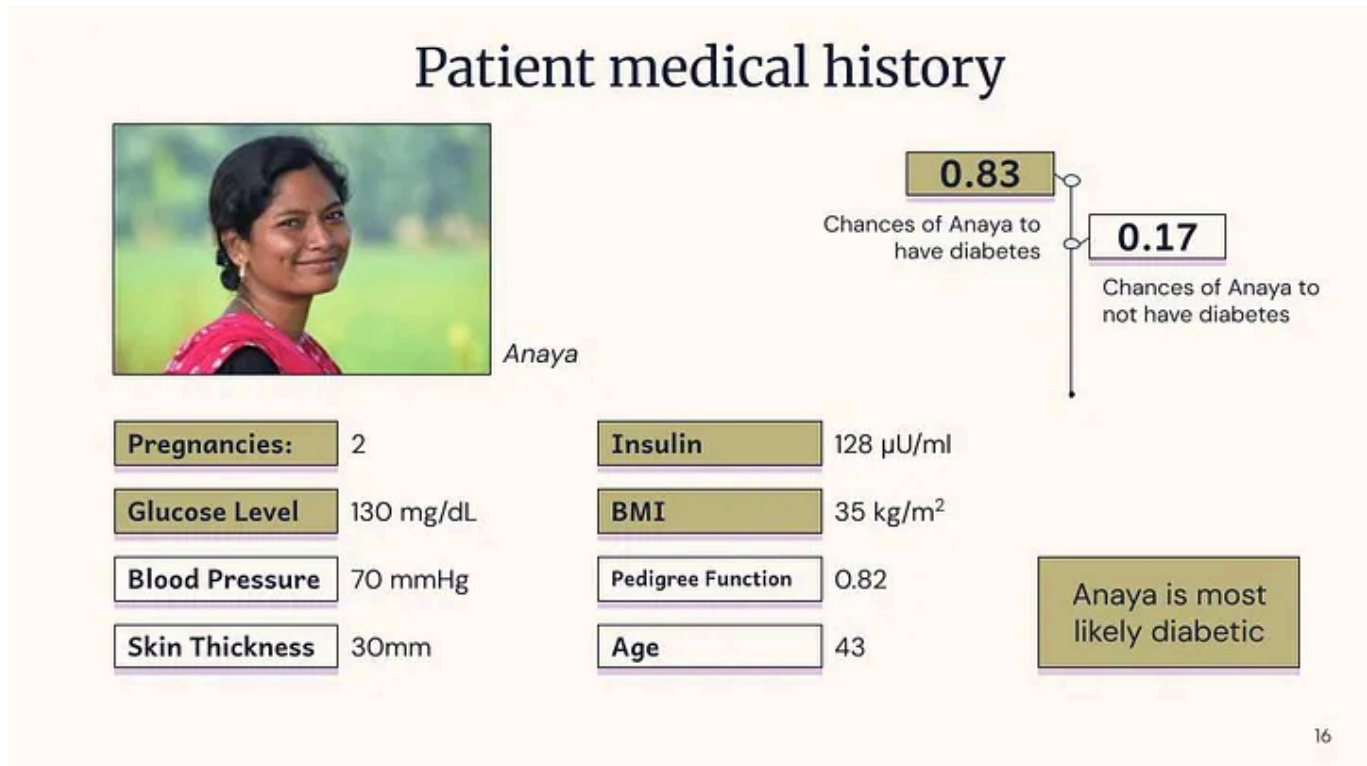
# Get the probability of each class
prediction_proba = best_rs_forest_model.predict_proba(scaled_patient_data)
print(f"Probability of no diabetes (Class 0): {prediction_proba[0][0]:.4f}")
print(f"Probability of diabetes (Class 1): {prediction_proba[0][1]:.4f}")
```

```
Prediction: The patient is likely to have diabetes.
Probability of no diabetes (Class 0): 0.1687
Probability of diabetes (Class 1): 0.8313
```

prediction from dummy data

This patient is predicted to be most likely diabetic — I call her Anaya

This is a dashboard that displays Anaya's user profile coupled with her medical history



anaya's profile

We also did an interactive one that asks for user input instead. We also collected medical input from a user (e.g., patient), preprocess the input to match the format used by a trained machine learning model (like KNN in this case), predict whether the patient is likely to have diabetes, and display the probability and interpretation of the result

We made use of a try-except validation block to be sure data is accepted in float data type only and appended to the list array `patient_data`

```
# Allow user input for prediction
def get_patient_input():
    """Gets patient attributes as input from the user with validation."""
```



```

feature_names = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', '
patient_data = []

print("Please enter the patient's details:")

for feature in feature_names:
    while True:
        try:
            value = float(input(f"Enter {feature}: "))
            # Add basic validation (e.g., non-negative values)
            if feature in ['Pregnancies', 'Age'] and value < 0:
                print("Value cannot be negative. Please enter again.")
                continue
            if feature in ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', '
                print("Value cannot be negative. Please enter again.")
                continue
            if feature == 'DiabetesPedigreeFunction' and value < 0:
                print("Value cannot be negative. Please enter again.")
                continue

            patient_data.append(value)
            break # Exit the loop if input is valid
        except ValueError:
            print("Invalid input. Please enter a numerical value.")
        except Exception as e:
            print(f"An error occurred: {e}")

    return np.array([patient_data])

# Get input from the user
user_patient_data = get_patient_input()

# Scale the user input data using the same scaler fitted on the training data
scaled_user_data = scaler.transform(user_patient_data)

user_prediction = knn.predict(scaled_user_data)

# Interpret the prediction
user_prediction_result = user_prediction[0]

if user_prediction_result == 1:
    print("\nPrediction: Based on the input data, the patient is likely to have di
else:
    print("\nPrediction: Based on the input data, the patient is likely not to hav

# Get the probability of each class
user_prediction_proba = knn.predict_proba(scaled_user_data)
print(f"Probability of no diabetes (Class 0): {user_prediction_proba[0][0]:.4f}")
print(f"Probability of diabetes (Class 1): {user_prediction_proba[0][1]:.4f}")

```

```
➡ Please enter the patient's details:  
Enter Pregnancies: 2  
Enter Glucose: 121  
Enter BloodPressure: 100  
Enter SkinThickness: 31  
Enter Insulin: 56  
Enter BMI: 30  
Enter DiabetesPedigreeFunction: 0.786  
Enter Age: 43  
  
Prediction: Based on the input data, the patient is likely to have diabetes.  
Probability of no diabetes (Class 0): 0.4400  
Probability of diabetes (Class 1): 0.5600
```

user input — prediction

Here, you can see that even if it predicts that the patient is likely to have diabetes, it is not as certain as the first example. So, who knows? This might turn out to be a false positive (or not)

As we continue to explore the intersection of health and machine learning, let's remember: every data point isn't just a statistic; it represents a human story.

All codes for this project can be found on my [GitHub](#)

Data Science

Machine Learning

Healthcare

Diabetes

Programming

**Written by Agbaoyeadejideji**

22 followers · 6 following

[Follow](#)

Responses (2)

**TheRed662**

What are your thoughts?

**Adegunju Mercy Ololade**

Jun 13



Amazing, good job man 🍌



11



1 reply

[Reply](#)**Joe Njenga**

Jun 22



Wow! I have been working on a similar project. Good job !



6

[Reply](#)

Recommended from Medium



 Big Programmer

Why `__init__.py` Changed the Way I Code in Python 😊💡

When I first started coding in Python, the file named `__init__.py` felt like a secret handshak...

May 19 🖱️ 680 💬 13  

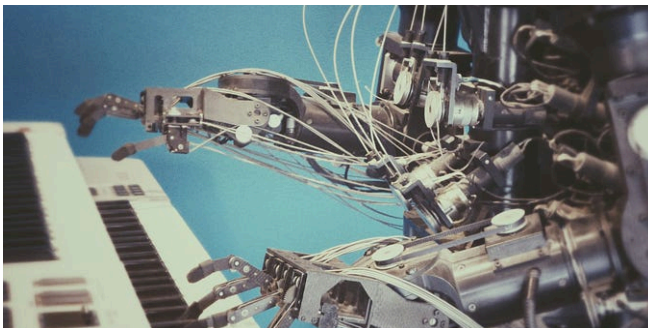


 In The Pub by Smillew Rahcuf

I'm 63—If You're Still in Your 40s (Or 50s), Read This

(it's a listicle)

🌟 May 29 🖱️ 10.2K 💬 251  



 In Utopian by Derick David

CEOs Are Literally Begging to Hire Anyone With This One Skill

This is the hottest AI job in 2025.



 In Artificial Intelligence in Plain ... by Kushagra Pa...

🧠 Predicting Diabetes Using Machine Learning: A Complete...

★ Jun 26

👤 2.3K

💬 118

🔖⁺

⋮



 In LearnAltoprofit.com by Raj Monetix 🎯

I Built a Side Hustle with AI. Now It Pays Me \$800 every month

\$800/Month from My Laptop (No BS)

★ Jun 27

👤 2K

💬 78

🔖⁺

⋮

In this article, we walk through a full machine learning pipeline for predicting diabetes usi...

★ Jun 21

👤 9

🔖⁺

⋮



 In AI Mind by Mr Tony Momoh

5 Industries AI Will Completely Take Over by 2026

It's Already Started

★ Jun 10

👤 4.7K

💬 217

🔖⁺

⋮

See more recommendations